

SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro: Centro de Diseño e Innovación Tecnológica Industrial

Ficha: 3235906

Programa de Formación: TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Nombre del Aprendiz: Laura Barona Saavedra

Nombre del Instructor: Andrés Felipe Parra Martínez

Fecha: 18 de agosto de 2026

INTRODUCCION

Hogar360 es una aplicación de gestión inmobiliaria compuesta por un frontend desarrollado en Angular y un backend distribuido en tres microservicios contruidos con Spring Boot (Java). El objetivo de este documento es explicar de forma clara y técnica cómo se establece la comunicación entre el frontend y el backend: dónde se configuran las direcciones de los servicios, qué mecanismo de autenticación se utiliza y cuál es el flujo completo que sigue una petición desde la interfaz hasta la respuesta del servidor. Además, se incluye una guía de evidencias visuales (pantallazos) para demostrar el funcionamiento de dicha conexión.

ARQUITECTURA GENERAL

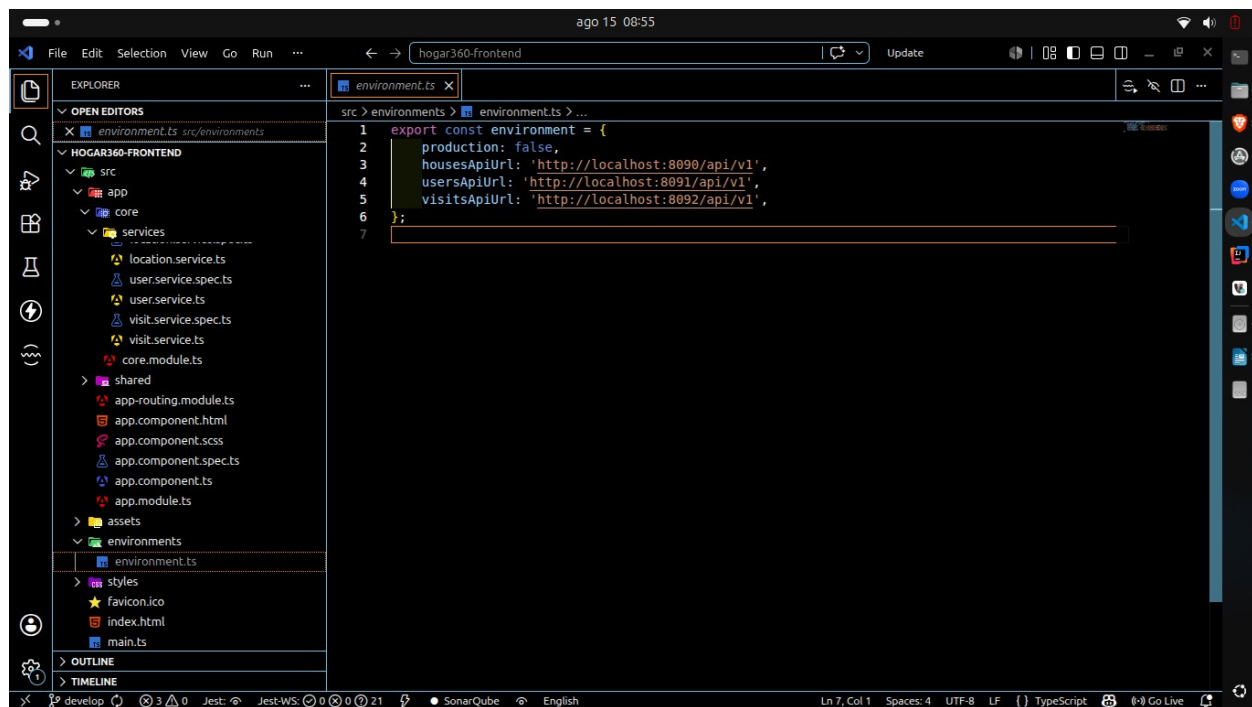
- Frontend: Angular 16 (carpeta hogar360-frontend), corre en `http://localhost:4200`
- Backend: 3 microservicios Spring Boot en Java:
- Houses API `http://localhost:8090` (propiedades, categorías, ubicaciones)
- Users API `http://localhost:8091` (autenticación JWT, usuarios, vendedores)
- Visits API `http://localhost:8092` (visitas, disponibilidad, reservas)
- Todos usan el prefijo de rutas `/api/v1`

DÓNDE SE DEFINE LA CONEXIÓN

Archivo: `src/environments/environment.ts`

```
export const environment = {  
  
  production: false,  
  
  housesApiUrl: 'http://localhost:8090/api/v1',  
  
  usersApiUrl: 'http://localhost:8091/api/v1',  
  
  visitsApiUrl: 'http://localhost:8092/api/v1',  
  
};
```

Aquí se guardan las URLs base de cada microservicio. Cualquier cambio de puerto o dominio del backend se hace en este archivo.



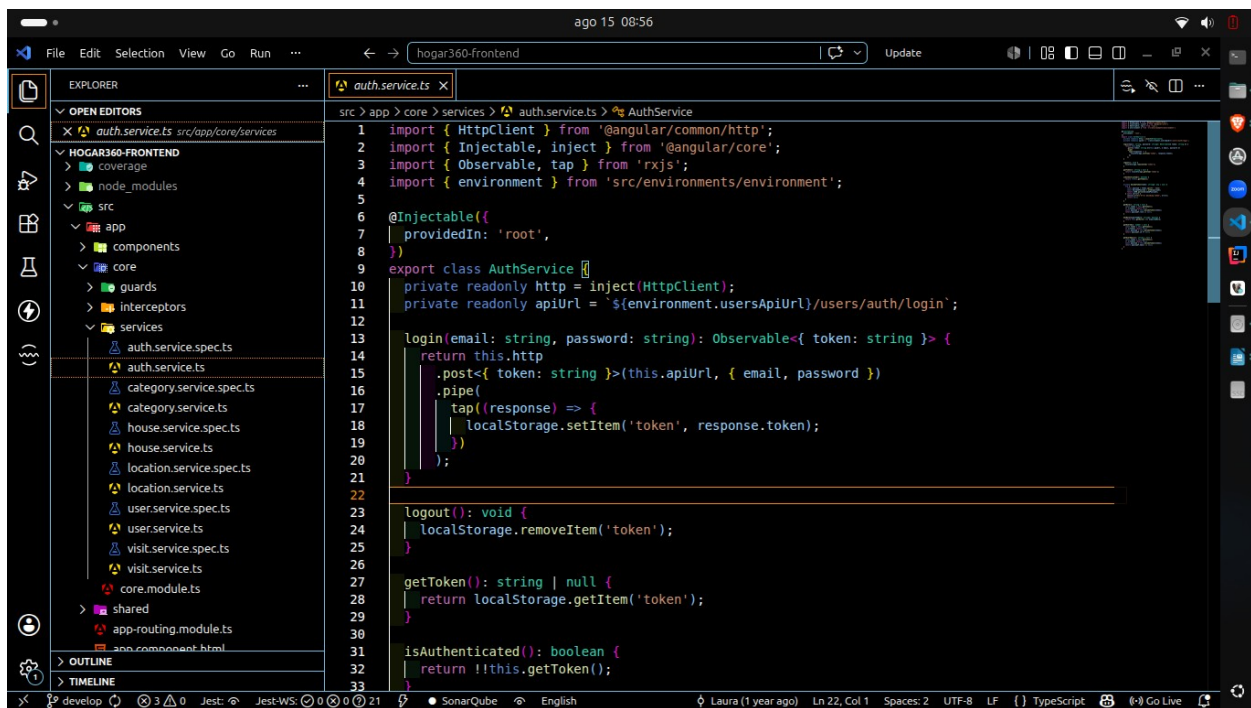
CÓMO SE REALIZA LA CONEXIÓN (FLUJO)

a) Los SERVICIOS inyectan HttpClient de Angular y concatenan la URL base

del environment con el endpoint del backend. Ejemplos:

- src/app/core/services/auth.service.ts

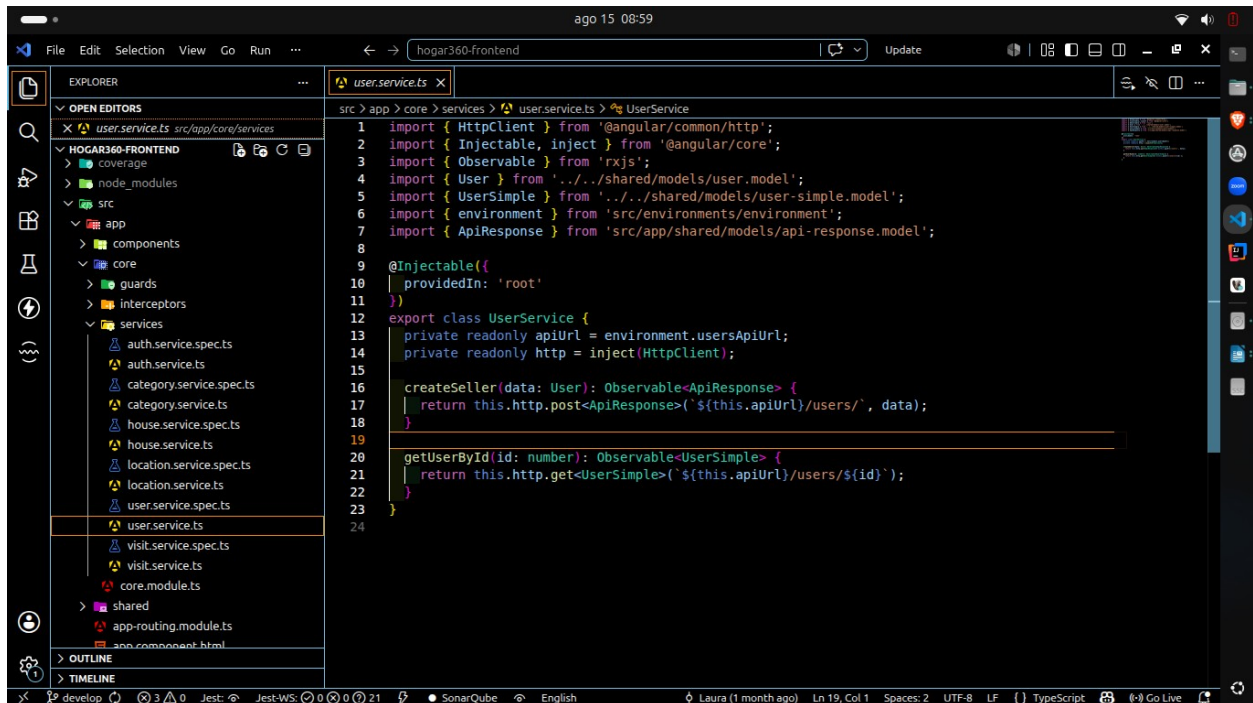
POST {usersApiUrl}/users/auth/login



```
1 import { HttpClient } from '@angular/common/http';
2 import { Injectable, inject } from '@angular/core';
3 import { Observable, tap } from 'rxjs';
4 import { environment } from 'src/environments/environment';
5
6 @Injectable({
7   providedIn: 'root',
8 })
9 export class AuthService {
10   private readonly http = inject(HttpClient);
11   private readonly apiUrl = `${environment.usersApiUrl}/users/auth/login`;
12
13   login(email: string, password: string): Observable<{ token: string }> {
14     return this.http
15       .post<{ token: string }>(this.apiUrl, { email, password })
16       .pipe(
17         tap((response) => {
18           localStorage.setItem('token', response.token);
19         })
20       );
21   }
22
23   logout(): void {
24     localStorage.removeItem('token');
25   }
26
27   getToken(): string | null {
28     return localStorage.getItem('token');
29   }
30
31   isAuthenticated(): boolean {
32     return !!this.getToken();
33   }
34 }
```

- src/app/core/services/user.service.ts

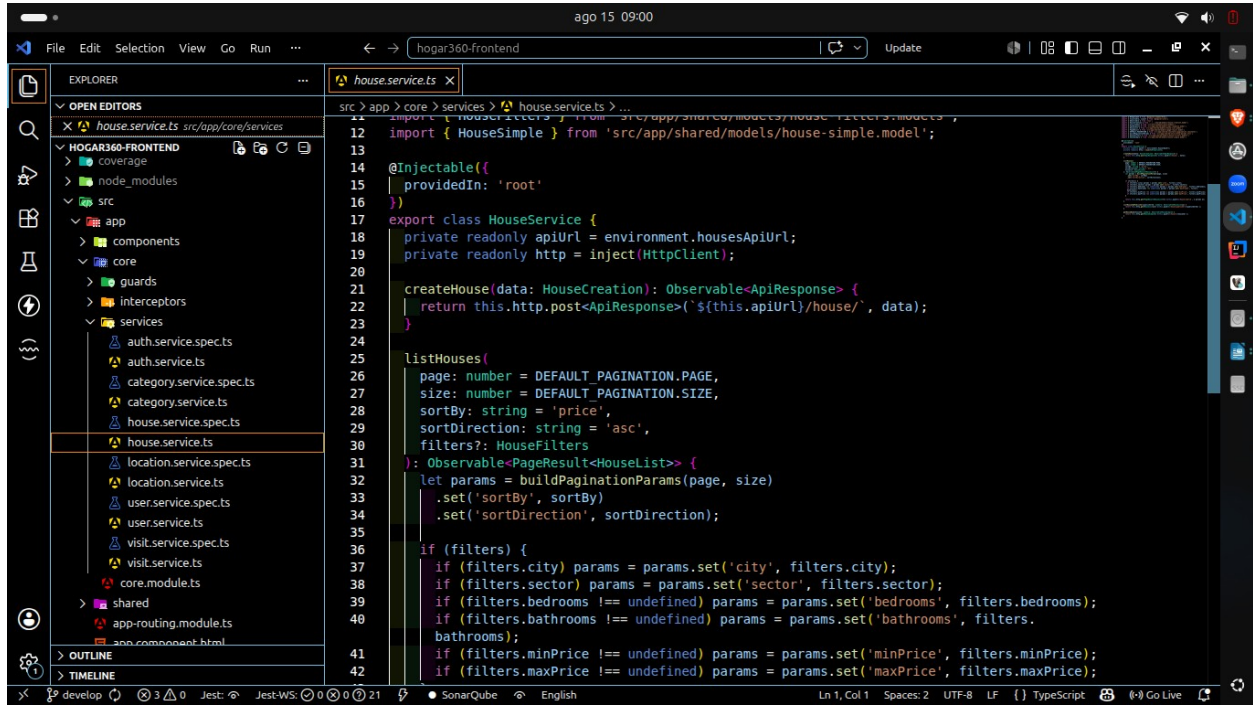
GET/POST {usersApiUrl}/users



```
1 import { HttpClient } from '@angular/common/http';
2 import { Injectable, inject } from '@angular/core';
3 import { Observable } from 'rxjs';
4 import { User } from 'src/app/shared/models/user.model';
5 import { UserSimple } from 'src/app/shared/models/user-simple.model';
6 import { environment } from 'src/environments/environment';
7 import { ApiResponse } from 'src/app/shared/models/api-response.model';
8
9 @Injectable({
10   providedIn: 'root'
11 })
12 export class UserService {
13   private readonly apiUrl = environment.usersApiUrl;
14   private readonly http = inject(HttpClient);
15
16   createSeller(data: User): Observable<ApiResponse> {
17     return this.http.post<ApiResponse>(`${this.apiUrl}/users/`, data);
18   }
19
20   getUserById(id: number): Observable<UserSimple> {
21     return this.http.get<UserSimple>(`${this.apiUrl}/users/${id}`);
22   }
23 }
24
```

- src/app/core/services/house.service.ts

CRUD {housesApiUrl}/house



- src/app/core/services/category.service.ts

CRUD {housesApiUrl}/category

```
1 import { HttpClient } from '@angular/common/http';
2 import { Injectable, inject } from '@angular/core';
3 import { Observable } from 'rxjs';
4 import { Category } from '../shared/models/category.model';
5 import { environment } from 'src/environments/environment';
6 import { PageResult } from '../shared/models/page-result.model';
7 import { DEFAULT_PAGINATION } from '../shared/constants/pagination.constants';
8 import { buildPaginationParams } from '../shared/utils/http-params.util';
9 import { ApiResponse } from 'src/app/shared/models/api-response.model';
10
11 @Injectable({
12   providedIn: 'root'
13 })
14 export class CategoryService {
15   private readonly apiUrl = environment.housesApiUrl;
16   private readonly http = inject(HttpClient);
17
18   createCategory(data: Category): Observable<ApiResponse> {
19     return this.http.post<ApiResponse>(`${this.apiUrl}/category/`, data);
20   }
21
22   getCategories(
23     page: number = DEFAULT_PAGINATION.PAGE,
24     size: number = DEFAULT_PAGINATION.SIZE
25   ): Observable<PageResult<Category>> {
26     const params = buildPaginationParams(page, size);
27     return this.http.get<PageResult<Category>>(`${this.apiUrl}/category/`, { params });
28   }
29 }
30
```

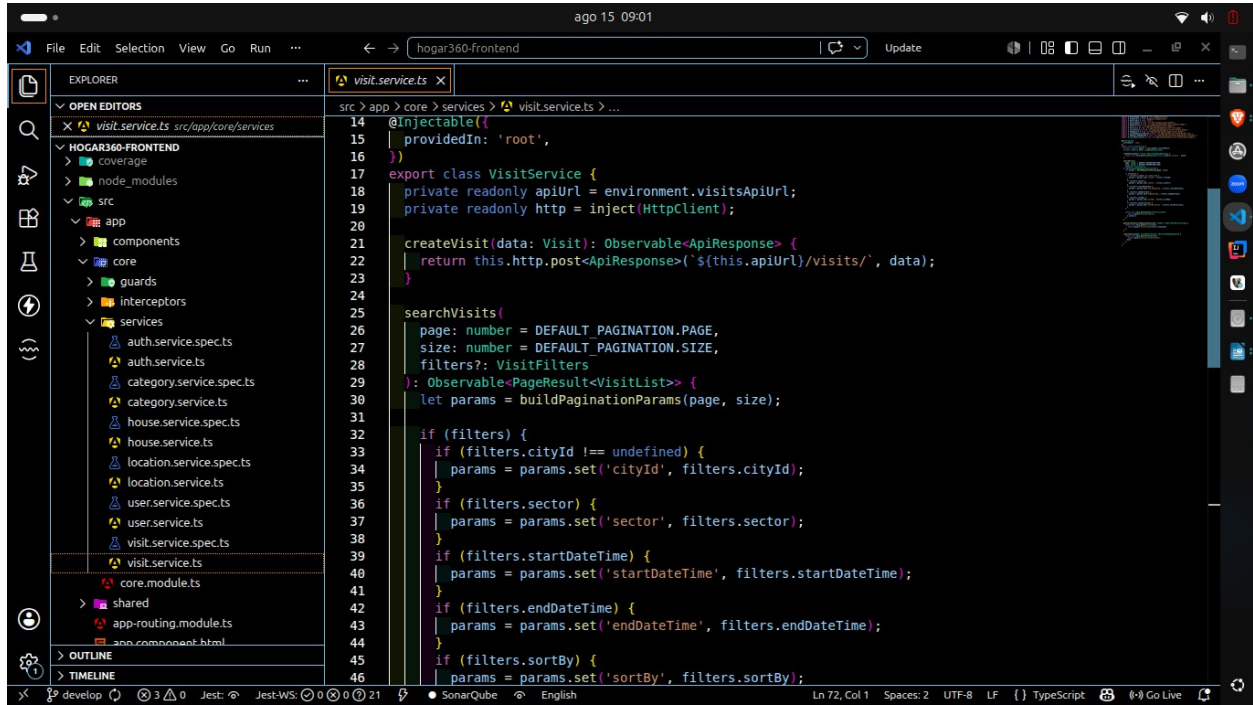
• src/app/core/services/location.service.ts

{housesApiUrl}/department, /city, /location

```
12 import { Observable } from 'rxjs';
13 import { ApiResponse } from 'src/app/shared/models/api-response.model';
14
15 @Injectable({
16   providedIn: 'root',
17 })
18 export class LocationService {
19   private readonly apiUrl = environment.housesApiUrl;
20   private readonly http = inject(HttpClient);
21
22   createLocation(data: Location): Observable<ApiResponse> {
23     return this.http.post<ApiResponse>(`${this.apiUrl}/location/`, data);
24   }
25
26   getDepartments(): Observable<Department[]> {
27     return this.http.get<Department[]>(`${this.apiUrl}/department`);
28   }
29
30   getCitiesByDepartment(departmentId: number): Observable<City[]> {
31     return this.http.get<City[]>(`${this.apiUrl}/city/department/${departmentId}`);
32   }
33
34   searchLocations(
35     searchTerm: string = '',
36     page: number = DEFAULT_PAGINATION.PAGE,
37     size: number = DEFAULT_PAGINATION.SIZE,
38     sortBy: string = 'city.name',
39     sortDirection: string = 'asc'
40   ): Observable<PageResult<LocationSearch>> {
41     let params = buildPaginationParams(page, size);
42     params = params
43       .set('searchTerm', searchTerm)
44       .set('sortBy', sortBy)
45     return this.http.get<PageResult<LocationSearch>>(`${this.apiUrl}/location/search`, { params });
46   }
47 }
48
```

- src/app/core/services/visit.service.ts

{visitsApiUrl}/visits, /visits/search, /visits/available, /visits/reservations



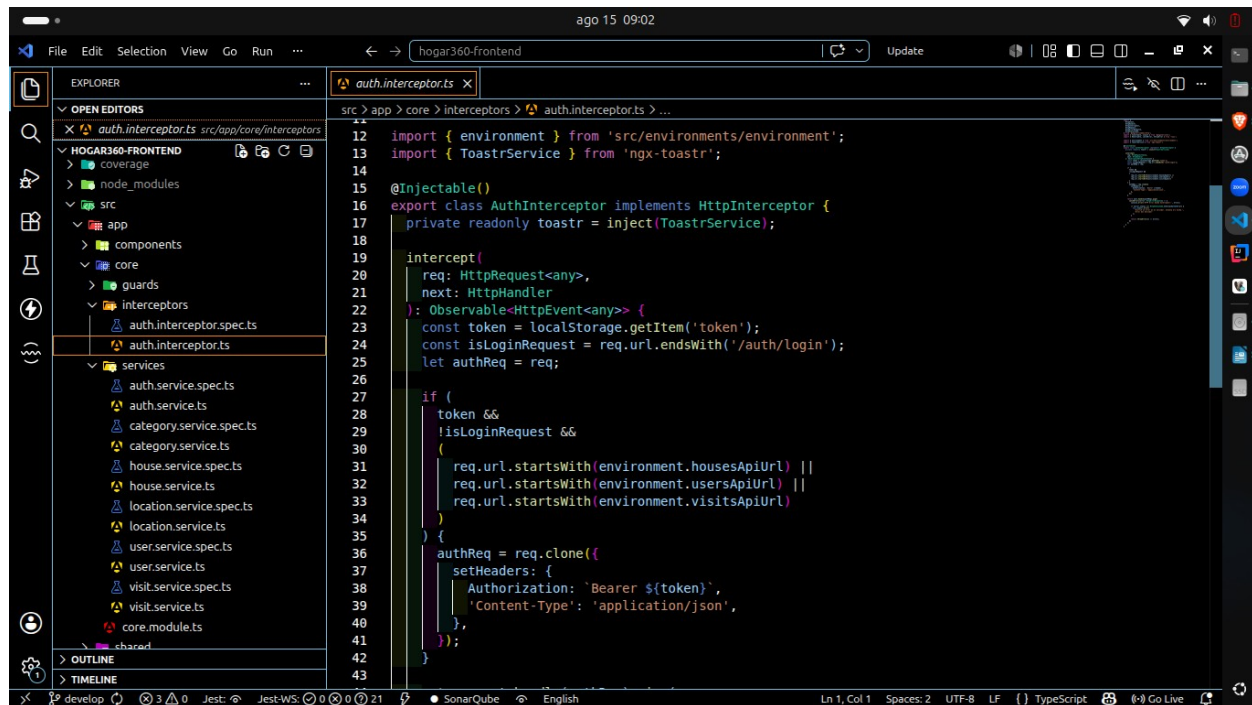
b) AUTENTICACIÓN JWT:

- El usuario hace login (auth.service.ts) y el backend responde con un token.
- El token se guarda en localStorage del navegador.
- El interceptor src/app/core/interceptors/auth.interceptor.ts intercepta

TODAS las peticiones HTTP y agrega el header:

Authorization: Bearer <token>

- Así el backend sabe qué usuario/rol está haciendo la petición.



c) RESPUESTAS:

- El backend responde en JSON. Los servicios devuelven Observable de RxJS y los componentes se suscriben para renderizar la información.

LÍNEA DE COMUNICACIÓN (RESUMEN)

Angular (componente) --> Servicio (HttpClient) --> Interceptor (agrega token) -->

environment.ts (URL base) --> Backend microservicio --> Respuesta JSON --> componente

renderiza

CONCLUSIÓN

La conexión entre el frontend y el backend en Hogar360 está centralizada y bien estructurada: las URLs base de los tres microservicios se definen en un único archivo de configuración (`environment.ts`), los servicios de Angular consumen esas rutas mediante `HttpClient` y el interceptor de autenticación garantiza que cada petición viaje con el token JWT requerido por el backend. Esta separación por microservicios y puertos permite que cada módulo (propiedades, usuarios y visitas) evolucione de forma independiente, y que la aplicación sea fácil de mantener y escalar. Gracias a este diseño, el frontend solo necesita conocer las URLs públicas de los servicios y el manejo del token, mientras que toda la lógica de negocio y seguridad permanece en el backend.